**Name: Soham Chavan**

**PRN: 124B1F022**

**Class: IT A**

# Assignment No. 5

## Aim:

To model the SwiftCargo transportation network as a directed, weighted multistage graph and to implement a backward Dynamic Programming algorithm that finds the minimum-cost delivery route from the source warehouse to the final delivery point, while handling real-time changes (traffic delays, road closures) and batch delivery requests.

## Theory:

### Multistage Graph

A multistage graph is a directed graph in which the vertices are partitioned into disjoint stages V1, V2, …, Vk such that every edge goes from a vertex in one stage to a vertex in the next stage. Each edge carries a cost (distance, time or fuel). The goal is to find the shortest (or sometimes longest) path from the source in V1 to the sink in Vk. Every path from source to sink passes through exactly one vertex of each intermediate stage.

- Structure: V1 = {A} (source), V2 = {B, C, D, E}, V3 = {F, G, H}, V4 = {I, J, K}, V5 = {L} (sink).
- Edges are directed and connect only Vi to Vi+1, so the graph has no cycles.
- Multistage property: the shortest path to a vertex in a stage depends only on the shortest paths to its predecessors in the previous stage (principle of optimality).
- The final answer is the shortest path to the sink L.

### Use Cases of Multistage Graphs in Real Life

- Logistics: routing parcels through warehouses and hubs (as in SwiftCargo).
- Speech recognition: word sequence prediction.
- Manufacturing: sequential processes with cost or time optimization.
- Network routing: data packet transmission in layered protocols.
- Finance: investment planning over time periods.

### Dynamic Programming (DP)

Dynamic Programming breaks a complex problem into smaller overlapping subproblems, solves each subproblem only once and stores its result. It requires two key properties: (1) Optimal substructure, i.e. the solution of the main problem can be built from solutions of subproblems, and (2) Overlapping subproblems, i.e. the same subproblems are needed again and again.

DP is better than plain recursion here because it avoids recomputation, works bottom-up (tabulation), and saves time and memory.

| Strategy | Suitable When | Pros | Cons |
|---|---|---|---|
| Greedy | Local choice leads to global optimum | Fast, simple | May give suboptimal solution |
| Dynamic Programming | Optimal substructure and overlapping subproblems | Guarantees optimal, efficient | More memory, needs planning |
| Backtracking | Combinatorial problems with constraints | Finds all/optimal combinations | Slow for large inputs |

**Why DP for this problem**

- The graph is acyclic.
- The best path from node i depends on the best paths from nodes in the next stage (optimal substructure).
- DP avoids redundant calculations by storing partial results.
- It is efficient for large multistage networks with thousands of nodes.

**Working of Backward DP in a Multistage Graph**

1. Number the stages 1 to N.
2. Initialize cost[destination] = 0.
3. Move backward stage by stage; for each node i, cost[i] = min over successors j of (weight(i, j) + cost[j]).
4. Store the chosen successor in the next[] array to track the path.
5. At the end, cost[source] gives the minimum cost and the path is reconstructed using next[].

**Algorithm**

```
Algorithm MultiStageDP(G)
1. cost[destination] = 0
2. for i = n-1 downto 1:
3.     cost[i] = infinity
4.     for each edge (i, j):
5.         if cost[i] > weight(i, j) + cost[j]:
6.             cost[i] = weight(i, j) + cost[j]
7.             next[i] = j
8. print cost[source] and the path
```

**Time and Space Complexity**

| Measure | Complexity / Description |
|---|---|
| Time | O(V + E) |
| Space | O(V) |
| Suitable for | Large multistage logistics graphs |

## Code:

```
import java.util.*;
```

```
public class MultistageGraphDP {
    static final int INF = 1_000_000;
    static int n;
    static String[] label;
    static int[] stageOf;
    static List<List<int[]>> graph = new ArrayList<>();

    static void init(String[] names, int[] stages) {
        n = names.length;
        label = names;
        stageOf = stages;
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
    }

    static void addEdge(int u, int v, int weight) {
        if (stageOf[v] != stageOf[u] + 1)
            throw new IllegalArgumentException("Edge must go from stage i to stage
i+1");
        graph.get(u).add(new int[]{v, weight});
    }

    static void updateWeight(int u, int v, int newWeight) {
        for (int[] e : graph.get(u)) if (e[0] == v) e[1] = newWeight;
    }

    static void closeRoad(int u, int v) {
        updateWeight(u, v, INF);
    }

    // Backward DP: nodes are numbered stage by stage, so going from n-1 down to 0
    // processes stages K-1, K-2, ..., 1.
    static void solve(int sink, int[] cost, int[] next) {
        Arrays.fill(cost, INF);
        Arrays.fill(next, -1);
        cost[sink] = 0;
        for (int i = sink - 1; i >= 0; i--) {
            for (int[] e : graph.get(i)) {
                int j = e[0], w = e[1];
                if (cost[j] < INF && w < INF && w + cost[j] < cost[i]) {
                    cost[i] = w + cost[j];
                    next[i] = j;
                }
            }
        }
    }

    static String path(int src, int[] next, int sink) {
        StringBuilder sb = new StringBuilder();
        int i = src;
        while (i != -1) {
            sb.append(label[i]);
            if (i == sink) break;
            sb.append(" -> ");
            i = next[i];
```

```
        }
        return sb.toString();
    }

    static void report(String title, int source, int sink) {
        int[] cost = new int[n];
        int[] next = new int[n];
        solve(sink, cost, next);
        System.out.println(title);
        System.out.println("  Minimum cost from " + label[source] + " to " +
label[sink] + " : " + cost[source]);
        System.out.println("  Optimal route : " + path(source, next, sink));
    }

    public static void main(String[] args) {
        // Stages: V1={A}, V2={B,C,D,E}, V3={F,G,H}, V4={I,J,K}, V5={L}
        String[] names = {"A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L"};
        int[] stages = {1, 2, 2, 2, 2, 3, 3, 3, 4, 4, 4, 5};
        init(names, stages);
        int A = 0, B = 1, C = 2, D = 3, E = 4, F = 5, G = 6, H = 7, I = 8, J = 9, K =
10, L = 11;

        addEdge(A, B, 9);  addEdge(A, C, 7);  addEdge(A, D, 3);  addEdge(A, E, 2);
        addEdge(B, F, 4);  addEdge(B, G, 2);  addEdge(B, H, 1);
        addEdge(C, F, 2);  addEdge(C, G, 7);
        addEdge(D, G, 11); addEdge(D, H, 11);
        addEdge(E, G, 8);  addEdge(E, H, 11);
        addEdge(F, I, 6);  addEdge(F, J, 5);
        addEdge(G, I, 4);  addEdge(G, J, 3);  addEdge(G, K, 6);
        addEdge(H, J, 6);  addEdge(H, K, 4);
        addEdge(I, L, 4);  addEdge(J, L, 2);  addEdge(K, L, 7);

        int[] cost = new int[n];
        int[] next = new int[n];
        solve(L, cost, next);

        System.out.println("Backward DP table (cost to reach sink L from each node):\
n");
        for (int i = 0; i < n; i++) {
            String nx = next[i] == -1 ? "-" : label[next[i]];
            System.out.println("Node " + label[i] + " (Stage " + stages[i] + ") : cost
= " + cost[i] + ", next = " + nx);
        }
        System.out.println("\nMinimum delivery cost from A to L: " + cost[A]);
        System.out.println("Optimal Route: " + path(A, next, L));

        System.out.println("\n--- Real-time update 1: traffic jam on road G -> J (3 to
12) ---");
        updateWeight(G, J, 12);
        report("After traffic update:", A, L);

        System.out.println("\n--- Real-time update 2: road closure F -> J and H -> K
---");
        closeRoad(F, J);
```

```
        closeRoad(H, K);
        report("After road closures:", A, L);

        System.out.println("\n--- Batch processing: several delivery requests, one DP
pass ---");
        solve(L, cost, next);
        int[] sources = {A, B, C, D, E, F};
        for (int s : sources) {
            System.out.println("Request from " + label[s] + " to " + label[L] + " :
cost = " + cost[s]
                    + ", route = " + path(s, next, L));
        }
    }
}
```

## Output:

```
Backward DP table (cost to reach sink L from each node):

Node A (Stage 1) : cost = 15, next = E
Node B (Stage 2) : cost = 7, next = G
Node C (Stage 2) : cost = 9, next = F
Node D (Stage 2) : cost = 16, next = G
Node E (Stage 2) : cost = 13, next = G
Node F (Stage 3) : cost = 7, next = J
Node G (Stage 3) : cost = 5, next = J
Node H (Stage 3) : cost = 8, next = J
Node I (Stage 4) : cost = 4, next = L
Node J (Stage 4) : cost = 2, next = L
Node K (Stage 4) : cost = 7, next = L
Node L (Stage 5) : cost = 0, next = -

Minimum delivery cost from A to L: 15
Optimal Route: A -> E -> G -> J -> L

--- Real-time update 1: traffic jam on road G -> J (3 to 12) ---
After traffic update:
  Minimum cost from A to L : 16
  Optimal route : A -> C -> F -> J -> L

--- Real-time update 2: road closure F -> J and H -> K ---
After road closures:
  Minimum cost from A to L : 18
  Optimal route : A -> B -> H -> J -> L

--- Batch processing: several delivery requests, one DP pass ---
Request from A to L : cost = 18, route = A -> B -> H -> J -> L
Request from B to L : cost = 9, route = B -> H -> J -> L
Request from C to L : cost = 12, route = C -> F -> I -> L
Request from D to L : cost = 19, route = D -> G -> I -> L
Request from E to L : cost = 16, route = E -> G -> I -> L
Request from F to L : cost = 10, route = F -> I -> L
```

## Conclusion:

The SwiftCargo network was modelled as a five-stage directed graph (V1 to V5), and the backward Dynamic Programming algorithm computed the cost to reach the sink from every node in a single pass, giving a minimum cost of 15 along the route A -> E -> G -> J -> L. The time complexity is O(V + E) and the space complexity is O(V), so the method scales to thousands of cities and routes. When a traffic jam raised the cost of road G -> J, and when roads F -> J and H -> K were closed, the DP table was simply recomputed and the route changed correctly to A -> C -> F -> J -> L (cost 16) and then A -> B -> H -> J -> L (cost 18). Because the table holds the best cost from every node, a whole batch of delivery requests was answered from one DP pass, which makes the approach suitable for real-time rerouting and batch processing in logistics.